



Laravel

SESSION & VALIDATION



Contents

- Session
 - Retrieving Data
 - Storing Data
 - Flash Data
- Validation
 - Validation
 - Displaying The Validation Errors
 - Form Request Validation
 - Manually Creating Validators
 - Working With Error Messages



HOA SEN
UNIVERSITY

www.hoasen.edu.vn

SESSION



Configuration

- The session driver configuration option defines where session data will be stored for each request. Laravel ships with several great drivers out of the box:
 - **file** - sessions are stored in storage/framework/sessions.
 - **cookie** - sessions are stored in secure, encrypted cookies.
 - **database** - sessions are stored in a relational database.
 - **memcached / redis** - sessions are stored in one of these fast, cache based stores.
 - **array** - sessions are stored in a PHP array and will not be persisted.



Retrieving Data (1)

- There are two primary ways of working with session data
 - The global session helper
 - Request instance

```
public function show(Request $request, $id)
{
    $value = $request->session()->get('key');

    //
}
```



Retrieving Data (2)

```
$value = $request->session()->get('key', 'default');  
$value = $request->session()->get('key', function () {  
    return 'default';  
});
```

```
Route::get('home', function () {  
    // Retrieve a piece of data from the session...  
    $value = session('key');  
    // Specifying a default value...  
    $value = session('key', 'default');  
    // Store a piece of data in the session...  
    session(['key' => 'value']);  
});
```

global session
helper

```
$data = $request->session()->all();
```



Determining If An Item Exists In The Session

- To determine if an item is present in the session, you may use the **has** method. The **has** method returns **true** if the item is present and is not null:

```
if ($request->session()->has('users')) {  
    //  
}
```

- To determine if an item is present in the session, even if its value is null, you may use the **exists** method. The **exists** method returns true if the item is present:

```
if ($request->session()->exists('users')) {  
    //  
}
```



Storing Data

- To store data in the session, you will typically use the put method or the session helper:

```
// Via a request instance...  
$request->session()->put('key', 'value');
```

```
// Via the global helper...  
session(['key' => 'value']);
```

```
$request->session()->push('user.teams', 'developers');
```

- Retrieving & Deleting An Item:

```
$value = $request->session()->pull('key', 'default');
```



Flash Data

- Data stored in the session using this method will be available immediately and during the subsequent HTTP request. After the subsequent HTTP request, the flashed data will be deleted. Flash data is primarily useful for short-lived status messages:

```
$request->session()->flash('status', 'Task was successful!');
```

```
$request->session()->reflash();
```

```
$request->session()->keep(['username', 'email']);
```



Deleting Data

```
// Forget a single key...  
$request->session()->forget('key');  
// Forget multiple keys...  
$request->session()->forget(['key1', 'key2']);  
// remove all data from the sessi  
$request->session()->flush();
```

- Regenerating The Session ID

```
$request->session()->regenerate();
```



Session Blocking

- By default, Laravel allows requests using the same session to execute concurrently.

```
Route::post('/profile', function () {  
    //  
})->block($lockSeconds = 10, $waitSeconds = 10)  
  
Route::post('/order', function () {  
    //  
})->block($lockSeconds = 10, $waitSeconds = 10)
```



HOA SEN
UNIVERSITY

www.hoasen.edu.vn

VALIDATION



Validation

```
$validatedData = $request->validate([  
    'title' => ['required', 'unique:posts', 'max:255'],  
    'body' => ['required'],  
]);
```

```
$validatedData = $request->validateWithBag('post', [  
    'title' => ['required', 'unique:posts', 'max:255'],  
    'body' => ['required'],  
]);
```

Error name

- assign the **bail rule** to the attribute to stop running validation rules on an attribute after the first validation failure

```
$request->validate([  
    'title' => 'bailrequiredunique:posts|max:255',  
    'body' => 'required',  
    'publish_at' => 'nullable|date',  
]);
```



Displaying The Validation Errors

```
<h1>Create Post</h1>
@if ($errors->any())
    <div class="alert alert-danger">
        <ul>
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif
<!-- Create Post Form -->
```



The @error Directive

- You may also use the @error Blade directive to quickly check if validation error messages exist for a given attribute. Within an @error directive, you may echo the \$message variable to display the error message:

```
<label for="title">Post Title</label>
<input id="title" type="text" class="@error('title') is-invalid @enderror">
@error('title')
    <div class="alert alert-danger">{{ $message }}</div>
@enderror
```



Form Request Validation

- To create a form request class, use the **make:request** Artisan CLI command:
 - The generated class will be placed in the app/Http/Requests directory

```
php artisan make:request StoreBlogPost
```

```
public function rules() {  
    return [  
        'title' => 'required|unique:posts|max:255',  
        'body' => 'required',  
    ];  
}
```

```
public function store(StoreBlogPost $request) {  
    // The incoming request is valid...  
    // Retrieve the validated input data...  
    $validated = $request->validated();  
}
```



Adding After Hooks To Form Requests

```
public function withValidator($validator) {  
    $validator->after(function ($validator) {  
        if ($this->somethingElseInvalid()) {  
            $validator->errors()->add('field', 'Something is wrong with this field!');  
        }  
    });  
}
```



Manually Creating Validators (1)

- If you do not want to use the validate method on the request, you may create a validator instance manually using the Validator facade. The make method on the facade generates a new validator instance:

```
class PostController extends Controller {  
    public function store(Request $request) {  
        $validator = Validator::make($request->all(), [  
            'title' => 'required|unique:posts|max:255',  
            'body' => 'required',  
        ]);  
        if ($validator->fails()) {  
            return redirect('post/create')  
                ->withErrors($validator)  
                ->withInput();  
        }  
        // Store the blog post...  
    }  
}
```



Manually Creating Validators (2)

- If you would like to create a validator instance manually but still take advantage of the automatic redirection offered by the request's validate method, you may call the validate method on an existing validator instance.

```
Validator::make($request->all(), [  
    'title' => 'required|unique:posts|max:255',  
    'body' => 'required',  
])->validate();
```

```
Validator::make($request->all(), [  
    'title' => 'required|unique:posts|max:255',  
    'body' => 'required',  
])->validateWithBag('post');
```

- If you would like to create a validator instance manually but still take advantage of the automatic redirection offered by the request's validate method, you may call the validate method on an existing validator instance.



Manually Creating Validators (3)

- If you have multiple forms on a single page, you may wish to name the MessageBag of errors, allowing you to retrieve the error messages for a specific form. Pass a name as the **second argument** to **withErrors**:

```
return redirect('register') ->withErrors($validator, 'login');
```

- You may then access the named MessageBag instance from the \$errors variable:

```
{{ $errors->login->first('email') }}
```

```
$validator = Validator::make(...);  
$validator->after(function ($validator) {  
    if ($this->somethingElseInvalid()) {  
        $validator->errors()->add('field', 'Something is wrong with this field!'); }  
});  
if ($validator->fails()) {  
    //  
}
```

After Validation Hook



Working With Error Messages

- Retrieving the first error message for a field

```
$errors = $validator->errors();  
echo $errors->first('email');
```

- Retrieving all error message for a field

```
foreach ($errors->get('email') as $message) { }
```

- Retrieving all error message for a field

```
foreach ($errors->get('attachments.*') as $message) { }
```

- Retrieving all error message for all field

```
foreach ($errors->all()) as $message) { }
```

- Determining if messages exist for a field

```
if ($errors->has('email')) { // }
```



Custom Error Messages

```
$messages = [  
    'required' => 'The :attribute field is required.',  
];  
$validator = Validator::make($input, $rules, $messages);
```

- the `:attribute` placeholder will be replaced by the actual name of the field under validation. You may also utilize other placeholders in validation messages.

```
$messages = [  
    'same' => 'The :attribute and :other must match.',  
    'size' => 'The :attribute must be exactly :size.',  
    'between' => 'The :attribute value :input is not between :min - :max.',  
    'in' => 'The :attribute must be one of the following types: :values',  
];
```

```
$messages = [  
    'email.required' => 'We need to know your e-mail address!',  
];
```

[Accepted](#)

[Active URL](#)

[After \(Date\)](#)

[After Or Equal \(Date\)](#)

[Alpha](#)

[Alpha Dash](#)

[Alpha Numeric](#)

[Array](#)

[Bail](#)

[Before \(Date\)](#)

[Before Or Equal \(Date\)](#)

[Between](#)

[Boolean](#)

[Confirmed](#)

[Date](#)

[Date Equals](#)

[Date Format](#)

[Different](#)

[Digits](#)

[Digits Between](#)

[Dimensions \(Image Files\)](#)

[Distinct](#)

[Email](#)

[Ends With](#)

[Exclude If](#)

[Exclude Unless](#)

[Exists \(Database\)](#)

[File](#)

[Filled](#)

[Greater Than](#)

[Greater Than Or Equal](#)

[Image \(File\)](#)

[In](#)

[In Array](#)

[Integer](#)

[IP Address](#)

[JSON](#)

[Less Than](#)

[Less Than Or Equal](#)

[Max](#)

[MIME Types](#)

[MIME Type By File](#)

[Extension](#)

[Min](#)

[Not In](#)

[Not Regex](#)

[Nullable](#)

[Numeric](#)

[Password](#)

[Present](#)

[Regular Expression](#)

[Required](#)

[Required If](#)

[Required Unless](#)

[Required With](#)

[Required With All](#)

[Required Without](#)

[Required Without All](#)

[Same](#)

[Size](#)

[Sometimes](#)

[Starts With](#)

[String](#)

[Timezone](#)

[Unique \(Database\)](#)

[URL](#)

[UUID](#)

References

- <https://laravel.com/docs/8.x/validation>